

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Modelling Comparative Analysis
Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.* (BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A
Modelling Comparative Analysis Assignment

Code of Conduct:

I T Bhupal/192425243 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*bhupalreddy*

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Automobile Fault Diagnosis

1. Problem Statement & Objectives

Problem Statement: An automobile service center faces the challenge of rapidly and accurately identifying underlying vehicle faults based on a specific set of customer-reported or mechanically observed symptoms. These symptoms include engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. The service center requires a formalized diagnostic model to translate these surface-level symptoms into concrete mechanical or electrical faults, thereby streamlining the repair process and minimizing diagnostic errors.

Domain & Decision-Making Requirement: The domain is automotive diagnostics. The core decision-making requirement is the ability to intelligently map combinations of Boolean symptom states (present or absent) to specific failure modes (e.g., battery failure, coolant leak, engine wear). The system must utilize logical deduction to rule out impossible faults and isolate the most probable cause based on the provided evidence.

Objectives of the Modelling Exercise: The primary objective is to mathematically and computationally model this diagnostic process using four distinct knowledge representation paradigms: Production Rules, Propositional Logic, First-Order Logic (FOL), and Prolog. By implementing and contrasting these paradigms, the exercise aims to demonstrate the mechanics of forward and backward chaining, evaluate the expressiveness of each logic system, and ultimately determine the most suitable architecture for real-world fault diagnosis.

2. Domain Knowledge & Facts

To construct an effective diagnostic model, the raw, unstructured domain of automotive mechanics must be translated into discrete, logically manageable components.

- **Entities:** The primary entities in this domain include the 'Vehicle' (the subject of diagnosis), 'Symptoms' (the observable conditions), and 'Faults' (the underlying mechanical issues to be identified).
- **Attributes & Facts:** The specific observable facts (attributes) are the presence or absence of: engine_overheating, starting_failure, abnormal_noise, low_mileage, and warning_indicators.
- **Relationships & Conditions:** The domain relies on causal relationships where specific combinations of facts logically point to a fault. For example, the condition of both a starting failure and active dashboard warning indicators strongly correlates with a catastrophic electrical issue.
- **Conclusions (Target Faults):** Based on the domain analysis, the system aims to deduce the following conclusions: *Battery/Alternator Issue*, *Coolant System Leak*, *Starter Motor Degradation*, and *Spark Plug/Engine Wear*.

3. Production Rule Model

The Production Rule Model represents domain knowledge as a set of straightforward conditional statements known as IF-THEN rules. This model utilizes a simplified knowledge base consisting of condition-action pairs. In the context of our automobile diagnostic system, the "conditions" are the observed symptoms, and the "actions" (or conclusions) are the diagnosed faults.

Production Rules:

- **Rule 1:** IF starting_failure IS TRUE AND warning_indicators IS TRUE THEN Fault = *Battery/Alternator Issue*.

- Rule 2: IF engine_overheating IS TRUE AND warning_indicators IS TRUE THEN Fault = *Coolant System Leak*.
- Rule 3: IF starting_failure IS TRUE AND abnormal_noise IS TRUE THEN Fault = *Starter Motor Degradation*.
- Rule 4: IF low_mileage IS TRUE AND abnormal_noise IS TRUE THEN Fault = *Spark Plug/Engine Wear*.

Deriving Conclusions: The inference engine in a production system continuously matches the IF portions of the rules against the current working memory (the facts). If a customer reports that their car won't start (starting_failure = TRUE) and there is a clicking sound (abnormal_noise = TRUE), the engine scans the rule base, finds that the conditions for Rule 3 are perfectly met, and "fires" the rule, deriving the conclusion that the starter motor is degraded.

4. Propositional Logic Model

Propositional Logic abstracts the domain into mathematical propositions—statements that evaluate strictly to True or False. This model eliminates the English-like syntax of production rules in favor of logical connectives like AND ($\wedge$), OR ($\vee$), NOT ($\neg$), and IMPLIES ($\rightarrow$).

Propositions: Let S_1 = Starting Failure, S_2 = Warning Indicators, S_3 = Engine Overheating, S_4 = Abnormal Noise, S_5 = Low Mileage. Let F_1 = Battery Issue, F_2 = Coolant Leak, F_3 = Starter Motor Degradation, F_4 = Engine Wear.

Logical Connectives and Inference Statements:

- $(S_1 \wedge S_2) \rightarrow F_1$
- $(S_3 \wedge S_2) \rightarrow F_2$
- $(S_1 \wedge S_4) \rightarrow F_3$
- $(S_5 \wedge S_4) \rightarrow F_4$

5. First-Order Logic Model

First-Order Logic (FOL) significantly increases the expressiveness of the model by introducing predicates, variables, and quantifiers (Universal $\forall$ and Existential $\exists$). Unlike propositional logic, which treats entire sentences as atomic facts, FOL can express properties of distinct entities and relationships between them.

Predicates:

- $\text{Symptom}(x, y)$: Represents that vehicle x exhibits symptom y .
- $\text{Diagnosis}(x, z)$: Represents that vehicle x is diagnosed with fault z .

Rules using Predicates and Quantifiers:

- $\forall x [\text{Symptom}(x, \text{starting_failure}) \wedge \text{Symptom}(x, \text{warning_indicators}) \rightarrow \text{Diagnosis}(x, \text{battery_issue})]$
- $\forall x [\text{Symptom}(x, \text{engine_overheating}) \wedge \text{Symptom}(x, \text{warning_indicators}) \rightarrow \text{Diagnosis}(x, \text{coolant_leak})]$
- $\forall x [\text{Symptom}(x, \text{starting_failure}) \wedge \text{Symptom}(x, \text{abnormal_noise}) \rightarrow \text{Diagnosis}(x, \text{starter_issue})]$
- $\forall x [\text{Symptom}(x, \text{low_mileage}) \wedge \text{Symptom}(x, \text{abnormal_noise}) \rightarrow \text{Diagnosis}(x, \text{engine_wear})]$

By using the universal quantifier ($\forall x$), the model dynamically applies these diagnostic rules to *any* specific vehicle introduced into the system, representing a major leap in scalability and relational reasoning over basic propositional logic.

6. Prolog Model & Implementation

Prolog natively implements a subset of First-Order Logic using Horn clauses. It allows us to directly translate the theoretical FOL predicates into executable code. The system utilizes dynamic memory allocation to store facts at runtime and uses recursive rules to infer diagnoses.

Relevant Prolog Code:

Prolog

```
% Dynamic declaration allows facts to be asserted at runtime
:- dynamic symptom/1.

% Knowledge Base Rules
fault(battery_issue) :- symptom(starting_failure), symptom(warning_indicators).
fault(coolant_leak) :- symptom(engine_overheating), symptom(warning_indicators).
fault(starter_issue) :- symptom(starting_failure), symptom(abnormal_noise).
fault(engine_wear) :- symptom(low_mileage), symptom(abnormal_noise).
fault(unknown) :- \+ fault(battery_issue), \+ fault(coolant_leak), \+ fault(starter_issue).

% Implementation Query Logic
diagnose_vehicle :-
    retractall(symptom(_)),
    assert(symptom(starting_failure)), % Simulating user input
    assert(symptom(warning_indicators)), % Simulating user input
    fault(Diagnosis),
    write('Diagnosed Fault: '), write(Diagnosis), nl.
```

```
101 ?- consult(['c:/Users/thand/OneDrive/Documents/AIES/lab/C05/AT3_implementation/diagnose_vehicle.pl']).
true.

101 ?- start.
=====
          Automobile Fault Diagnostic System
=====
Please answer yes. or no. to the following symptoms.
(Note: Ensure you include the period after your answer)

Does the vehicle exhibit starting_failure? (yes/no): yes.
Does the vehicle exhibit warning_indicators? (yes/no): |: no.
Does the vehicle exhibit engine_overheating? (yes/no): |: yes.
Does the vehicle exhibit abnormal_noise? (yes/no): |: yes.
Does the vehicle exhibit low_mileage? (yes/no): |: no.

--- DIAGNOSIS RESULT ---
Detected Fault: Starter Motor Degradation
=====
true.
```


7. Forward & Backward Chaining

Forward Chaining (Data-Driven Reasoning):

In forward chaining, the inference engine starts with the known facts and continuously applies rules to generate new facts until a goal is reached.

- *Sequence:* The mechanic inputs the facts: 1. starting_failure is TRUE. 2. abnormal_noise is TRUE. The system scans all rules. It sees the IF conditions for Rule 3 are fully satisfied by the working memory. It "fires" Rule 3 and adds a new fact to memory: Fault = starter_issue.

Backward Chaining (Goal-Driven Reasoning):

Prolog natively uses backward chaining. The system starts with a hypothesis (the goal) and works backward to see if the available evidence supports it.

- *Sequence:* The system asks, "Is the goal fault(coolant_leak) true?" To prove this, it checks the rule requirements: it needs to prove symptom(engine_overheating) and symptom(warning_indicators). It checks the database of facts. If the mechanic has not entered engine_overheating, the goal fails. The system then backtracks and tries the next goal, fault(starter_issue), repeating the verification process until a hypothesis is proven true.

8. Comparative Analysis

Parameter	Production Rules	Propositional Logic	First-Order Logic (FOL)	Prolog
Knowledge Representation	IF-THEN condition-action pairings.	Atomic propositions linked by mathematical operators.	Predicates, variables, constants, and quantifiers.	Horn clauses consisting of facts and rules.
Expressiveness	Low to Medium. Highly rigid	Very Low. Cannot represent	High. Can express complex relations between objects.	High. A practical computational

Parameter	Production Rules	Propositional Logic	First-Order Logic (FOL)	Prolog
	syntax.	relationships or generalize.		implementation of FOL.
Rule Representation	Intuitive, English-like conditional statements.	Strict mathematical formulas and truth tables.	Complex logical formulas using universal/existential quantifiers.	Predicates where the Head is the conclusion and the Body contains conditions.
Inference Mechanism	Pattern matching (e.g., Rete algorithm).	Modus Ponens, Resolution, Truth Table evaluations.	Unification, Generalized Modus Ponens, Resolution.	Unification, Backward Chaining, and Depth-First Search.
Forward Chaining	Highly supported. Natural fit for data-driven systems.	Supported via structural mathematical deduction.	Supported, though computationally heavy.	Can be implemented manually, but not native.
Backward Chaining	Supported, though secondary to forward matching.	Supported (starting from conclusion to premises).	Supported mathematically.	Natively supported. The core engine of Prolog.
Handling Relationships	Poor. Rules must be duplicated	None. Treats facts as isolated,	Excellent. Natively links variables to distinct objects.	Excellent. Connects logic through

Parameter	Production Rules	Propositional Logic	First-Order Logic (FOL)	Prolog
	for different entities.	atomic variables.		unified variables.
Scalability	Low. Rule bases become "spaghetti" logic as they grow.	Lowest. Combinatorial explosion of rules required.	High theoretically, but computationally expensive to process.	Medium to High. Highly dependent on efficient rule ordering.
Explainability	High. Easy to trace which IF condition fired a THEN action.	Low. Tracing abstract math formulas is difficult for humans.	Medium. Logical traces are robust but mathematically dense.	High. Inference traces map perfectly to human logical deduction.
Ease of Implementation	Very High. Easy for non-programmers to write basic rules.	Medium. Requires understanding of discrete mathematics.	Low. Highly abstract; difficult to build a custom engine from scratch.	High (for developers). Built-in engine handles all inference algorithms.
Suitability for the Problem	Good for very small, localized diagnostic trees.	Poor. Too rigid for a dynamic automotive garage environment.	Good theoretically, but lacks a ready-to-use execution environment.	Excellent. Perfect for rapid prototyping of goal-driven expert systems.

9. Results, Discussion & Recommended Model

Results Analysis: The modeling exercise successfully demonstrated that automobile fault diagnosis can be mathematically formulated across multiple paradigms. Propositional logic proved too rigid, lacking the relational depth required to differentiate between multiple vehicles in a bustling service center. While First-Order Logic provided the necessary expressiveness, it remained largely theoretical without an underlying execution engine.

Recommended Model: Prolog is unambiguously the most appropriate approach for this real-world problem. Because automobile diagnostics is inherently a goal-driven process (a mechanic sees a symptom and hypothesizes a fault, working backward to verify it), Prolog's native backward-chaining engine aligns perfectly with the cognitive workflow of the domain experts. Furthermore, Prolog handles the heavy computational lifting of unification and backtracking automatically, drastically reducing development time.

Limitations and Improvements: The primary limitation of the Prolog model is its reliance on crisp, binary logic; an engine is either overheating (True) or it is not (False). In reality, automotive symptoms exist on a spectrum (e.g., slight overheating vs. critical failure). A major future improvement would involve transitioning the Prolog backend to incorporate Fuzzy Logic or Certainty Factors, allowing the system to process probabilistic inputs and output a ranked list of likely faults rather than a single absolute conclusion.

10. Conclusion, References & GitHub

Conclusion: The formal modeling of automobile fault diagnosis emphasizes the critical importance of selecting the correct knowledge representation paradigm. While Production Rules offer simplicity, they fail to scale. Propositional and First-Order logic provide robust mathematical foundations but lack native execution environments. Ultimately, Prolog synthesizes the expressiveness of First-Order Logic with an incredibly efficient backward-chaining inference engine, resulting in a highly effective, transparent, and scalable diagnostic expert system suitable for modern automotive service centers.

References:

1. Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Bratko, I. (2011). *Prolog Programming for Artificial Intelligence* (4th ed.). Pearson Education.
3. Negnevitsky, M. (2005). *Artificial Intelligence: A Guide to Intelligent Systems*. Addison-Wesley.

GitHub Repository:

https://github.com/bhupalreddythandlam/MLA0103-AIES/tree/main/CO5/AT3_implementation

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				